

Christoffer Nilsson

Developer and tech lead

I am the kind of person who really loves my job. I spend a lot of my free time programming applications and puzzles just for fun and reading blogs. Apart from my job I like to spend time with my family, my girlfriend who is a teacher and our 8-year-old son and 4-year-old daughter. I also follow a lot of sports for example golf, which I also (try to) play myself, hockey and some football.

It is important to me to have a good relationship with the people I work with. I like to collaborate daily with my colleagues and value team effort more than what I do myself. In teams where I have worked for longer time periods, I have ended up in a role where I handle much of the communication with stakeholders, architects and users. It is something I like since I value having a bigger picture of how my team fits in with the organization.

At Svenska Spel, I have grown into the tech lead and software architect role while helping establish a new team and modernize systems in the company-wide move to the cloud. The experience has been particularly formative for me, and the mix of modern technology and technical problems to solve is what drives me.

The project that I am most proud of having been a part of is the one at Thomas Cook. I worked in a very inspiring team and was trusted with a lot of responsibility even though I was the only consultant on the team. The journey we made, from a monolithic .NET application to more of a “microservice” architecture with a modern tech stack and continuous delivery, has shown me what is possible to achieve if you set your mind to it.

Experience

Svenska Spel

2023-08-15 - Ongoing

Scala · .NET · k8s · GCP · GitHub · Argo · GitHub Actions · Testkube · LGTM · Kafka · Kafka Streams · ksqlDB · Adobe Experience Platform · Data Mesh

Svenska Spel is Sweden’s state-owned gambling company. The company offers games and betting on the Swedish market and works for a healthy and safe gambling market.

Backend developer martech

For two years, I worked on integrating Svenska Spel’s marketing platform, Adobe Experience Platform, focusing on real-time communication and data supply from data products. Svenska Spel uses a Data Mesh strategy to share data. The events included transactions on the order of a few hundred events per second, which made guarantees of data correctness and latency major challenges, while the technology was new to many people in the project. We built real-time aggregates over streaming data with ksqlDB and Kafka Streams. .NET remains the company’s main language, while my team used Scala with Kafka Streams where relevant, for example for real-time processing of transactions on top of Kafka. The work also included handling personal data and GDPR. It resulted in a faster setup for new customer communication and the ability to test and measure its impact.

Backend developer pre-study responsible gaming

In parallel with the martech work, I took part in a pre-study for a similar assignment. An event or customer

behaviour would trigger an automated action, in this case, for example, identifying excessive gambling and trying to curb it through communication and restrictions. The initial plan was to also use Adobe Experience Platform for this, but it later became a more custom-built solution in Kafka Streams directly on our Data Mesh.

Tech lead responsible gaming

For the past year, I have worked as a tech lead, a role also titled software architect at Svenska Spel. I am continuing the work carried out during the pre-study, but in a significantly larger team with new internal and external hires. The role involves responsibility for designing the systems the team owns: architecture, planning, stakeholder dialogue, and working out requirements. In my view, the main challenges have been establishing a new team with new systems while we take ownership of existing systems and modernize them as part of Svenska Spel's company-wide programme to move from on-premise to the cloud (GCP). As part of the programme, we move code to GitHub, build with GitHub Actions, and deploy with Argo to Kubernetes. We use Testkube for integration tests and Config Connector (KCC) to configure databases and some permissions. The migration also brings a clearer microservice architecture. We are also responsible for running our services and setting up monitoring and alarms with Grafana and the rest of the LGTM stack.

[Memmo.me](#)

2021-08-16 - 2023-06-30

React · React Native · Node.js · GraphQL · Team lead · Payments · CMS/Admin · Real-time messaging · Search · Algolia · Terraform · GCP · GitHub · GitHub Actions

Memmo.me was a platform for ordering personalised video greetings from celebrities and influencers. Customers could request greetings and messages directly from the people they followed.

Full stack team lead

I joined Memmo.me when the company had just started expanding the IT team and quickly became a team lead for a team focusing on the B2C product and conversion. As a team lead I worked closely with a product owner and was responsible for planning the work for the team, planning with stakeholders and providing estimates, and coaching teammates to help them grow. Since the company was still small, I worked with almost everything in our codebase: a React Native app, a custom-built CMS/admin tool, payments and payouts, invoicing, real-time messaging, and integrations with customer support and search services.

[Netlight](#)

2014-10-15 - 2021-08-16

Consulting · Coaching · Recruitment · Mentoring

Netlight is a consultant company with about 1400 employees around Europe. It is a company that organically grows and has recruitment as a core function. Netlight values the individual and is organized as a network rather than being hierarchical.

During my years at Netlight I grew with the company values. We always had a lot of focus on coaching, which I still see in myself as I work with clients. Working in a team is something that is really important to me, and

I always strive to help my teammates develop and enable them to fulfil their goals.

Global Connect/IP-Only - Digitization

via Netlight

2018-12-01 - 2021-08-16

Java · Spring Boot · Docker Swarm · .NET · C# · Octopus · GitLab · TeamCity · MariaDB · MS SQL · Kibana · Elasticsearch · Grafana · AngularJS · TypeScript · Nginx · ArchiMate · RabbitMQ · Jest · Python · JUnit

Global Connect/IP-Only builds and operates one of Sweden's largest and neutral fiber networks. Their business model is to empower digitization by owning, developing and offering the best infrastructure and low level services for the digital society.

The focus of the project was to develop both customer-facing and internal IT systems. Before the project started, different parts of the company worked mostly in their own defined processes and tools. The tools could be purchased systems, but Excel and email were mostly used to handle sales and deliveries. The aim was to document all core processes across the different parts of the company, set up integrations and build new systems to make the work faster, easier and less error-prone.

System developer .NET

I started the project as a backend .NET developer working with business sales support. IP-Only had just merged with another company called DGC and had inherited some .NET systems for searching connectivity options for addresses and some tools used for deliveries in the network acquired from DGC. The main issues with this project were that these systems were in bad shape and lacked documentation. Quickly after starting the project I raised the issues and proposed building the new requirements in the IP-Only stack and developing a new module in the legacy system to benefit from everything done in the IP-Only tech stack. This finally led to so much of the functionality already being in the IP-Only stack that we decided to build the UI and missing logic there as well and deprecate the old system. The value gained from this was a more reliable system, and the entire IT development department had the competence needed to fix bugs and monitor the system instead of having a single person responsible for all of this as before.

Full stack developer Java/AngularJS

As the project progressed, the role became more and more that of a Java/AngularJS full-stack developer, continuing to build a customer portal. The goal was to let enterprise customers log in themselves and search for network options for their locations. We built this new customer-facing feature on the search engine described in the previous role. It had an Angular frontend for managing addresses and products. We integrated this system with a new self-developed product catalogue and a back-office UI to manage all products. The integration gave customers a faster way to get quotes, often with automatic replies, when buying new products.

Full stack developer Java/AngularJS

When the first version of the customer portal was released, I felt that I wanted to try something new, and the client agreed to move me from the sales support team to the operational support team. There I started working in a development team where much of the work depended on just one team member. I worked closely with the project manager to develop a working development process and implement most of the

Scrum model. This new role had a few different goals: setting up a back-office application for reserving and finding different forms of network identifiers; providing data for a new ServiceNow integration; and setting up APIs to fetch data from ServiceNow APIs. The other prioritised goal was unifying the stack used to monitor the two recently merged networks. This meant merging data sources and setting up new services to handle both networks. What I take away from this role is the soft skills I developed: coaching new team members and introducing stakeholders to sprint demos.

Thomas Cook NE - Responsive website redesign

via Netlight

2015-01-01 - 2018-06-30

React · Node.js · Redux · Redux-saga · Express · GraphQL · Apollo-client · Apollo-server · Jenkins · Drone · Kibana · Elasticsearch · MS SQL · Redis · RabbitMQ · TFS · .NET · C# · GitHub · Visual Studio · Enzyme · Mocha · Selenium

Thomas Cook Group was one of the world's largest organisers of charter trips. The group included, among others, the tour operators Ving, Spies, Tjäreborg and Globetrotter; the airline Thomas Cook Airlines Scandinavia; and the hotel chain Sunwing Family Resorts. At the time, Thomas Cook Group had 22,000 employees in 15 countries, 20 million customers and a turnover of almost SEK 90 billion.

Thomas Cook Northern Europe wanted reinforcement for their project group that worked on redesigning Ving.se, Ving.no, Spies and Tjäreborg to make the website more mobile-friendly. They wanted to make it responsive, more user-friendly and easier to understand for the convenience of their users.

Full stack developer

I was originally intended to be a back-end .NET developer, but I joined a team that had just built a Node.js API with an AngularJS front-end. Before starting their next project, I compared the new Angular version and React. To do this, I tried building a small application in both frameworks and talked to my Netlight colleagues at SVT who had used React for a while. I proposed switching to React, which we ended up doing, and it worked out great. I still feel that I like working in React better than Angular even with the newer versions. We then continued to build smaller services for their webpage using Node.js/Express APIs and React front-end apps.

Workshop leader

As we started to build more React applications for our external website, we noticed the need for a component library and common styles to use in our applications. I started pushing for us to take after, for example, Airbnb and make a style guide with reusable components, published both as an npm package and as a webpage with examples and code snippets. A colleague and I decided to bootstrap this with a workshop for all teams working on the web. We set up a basic repository, had a short introduction, divided people from different teams into groups and assigned each group one component to build, as well as an example page where it could be used. They then spread out and started working from their own ideas, while we as workshop leaders moved between groups to see if anyone needed help or input from us. The package was well received by all teams, and every developer was included at an early stage in creating the package and components. It was important to us that all web developers were comfortable making changes and adding new components to the library, since it would otherwise become more of an obstacle than a helpful tool. When I left Thomas Cook, this project was still going strong and had a large impact on the pace at which new applications could be developed while still keeping a coherent UI.

Full stack developer GraphQL

I was trusted with more and more responsibilities at Thomas Cook, and when a colleague and I started to discuss the potential of setting up a GraphQL API for the web, we got the OK to test it on a small application. The reason to try it was that we had seen that much time went into writing APIs for similar data again and again across the company, for example text data from our CMS and facts about hotels. We also needed to figure out how to handle caching for our APIs to make API calls run faster when possible.

We started off as a team of three and set up our own framework for fetching data from different sources such as caches, APIs and databases. This was not available out of the box in Apollo Server at the time we built it, although much of it is today. We set up our first version of caching using an MS SQL in-memory database as a key-value store. We wanted to use Redis, but there were hosting issues that needed sorting out beforehand. We therefore used our SQL database in the same way as we would use Redis, with the same kind of methods available in Redis, to make it easier to swap them out later. Towards the end of the project, we also got access to a Redis instance and could make the switch.

The new API made using data provided by services built in other teams easier, while also providing us with useful data on response times for different types of data. We also tried setting up Apollo Client directly on top of React, but found it was not easy to build the same functionality that we were used to building with Redux and redux-saga. So we ended up using it in our client data-access layer and keeping our shared application data in Redux as it had been previously.

Education

Royal institute of technology - Master of science program for Computer science
2008-09-01 - 2014-09-01

Technical University of Munich - Exchange semester, computer science program
2013-09-01 - 2013-12-20

Amf1 Berga - Military service, combat boat driver
2009-09-01 - 2010-06-20